EE7218 / EC7207 — High Performance Computing Project

Parallel Training of an MLP Neural Network for Protein Secondary Structure Classification

A CB513 Evaluation Using Serial, OpenMP, POSIX Threads, MPI, Hybrid MPI+OpenMP, and CUDA

Group 36

EG/2021/4684 MUNSIF M.F.A.
EG/2021/4685 MUTHUKUMARI H.M.S.
EG/2021/4686 NADHA M.J.

Abstract

We implement and evaluate six variants of the training loop for a compact multi-layer perceptron (MLP) that classifies per-residue secondary structure (Q_3) on the CB513 benchmark: a serial baseline, two shared-memory parallelisations (OpenMP and POSIX threads), a distributed-memory implementation with MPI, a hybrid MPI + OpenMP version, and a CUDA implementation that offloads the whole training loop to the GPU. All variants share the same model, command interface, and data split so that accuracy can be compared directly. In the final 80-epoch run, the serial baseline reaches test $Q_3 = 59.28\,\%$ and acts as the reference target for every parallel variant. We measure per-epoch wall-clock time as a function of shared-memory thread count, MPI rank count, hybrid rank/thread layout, and CUDA batch size, then discuss where each design scales well and where it plateaus.

May 21, 2026

Contents

1 Introduction and Motivation

Protein secondary-structure prediction assigns each residue of a protein sequence to one of three classes: α-helix (H), β-sheet (E), or coil (C). It is an enabling step for downstream tertiary-structure analysis and drug design, and the classical approach is to train a neural network over a sliding window of residues centred on the target residue [3, 1]. Zhong et al. [1] showed on early multi-core Intel hardware that the training of such a network parallelises well using OpenMP or POSIX threads, obtaining speedups close to the core count. In this project we repeat that experiment on modern hardware *and* extend the parallelisation strategies to distributed memory (MPI), a hybrid shared/distributed scheme, and GPU acceleration (CUDA).

The report focuses on three evaluation questions: how each parallel programming concept is applied to the neural-network training loop, whether the parallel versions preserve the serial prediction accuracy, and how runtime changes as the thread count, rank count, hybrid layout, or CUDA batch size is varied. CUDA is included as an additional GPU implementation because the final experiments were completed for it.

2 Dataset and Preprocessing

We use the CB513 benchmark of Cuff & Barton [2], a curated set of 513 non-homologous protein domains (∼84 000 residues). The processing pipeline is deterministic under a fixed seed:

1. **Download** — fetches the canonical distribution tarball.
2. **Parse** — extracts `(protein_id, residue, 8-state-label)` tuples from the raw `.all` files.
3. **8→3 collapse** — DSSP labels are collapsed to Q3 using the mapping described in [1]: $H, G, I \to H$; $B, E \to E$; everything else $\to C$.
4. **Sliding window** of size 13 with flat-edge padding, one-hot encoded over the 20 amino-acid classes, giving an input vector of dimension $20 \times 13 = 260$.
5. **Split** — fixed 70/15/15 train/validation/test; split indices are persisted with the processed dataset metadata.
6. **Binary persistence** — the final tensors are serialised under the processed CB513 binary-data directory for fast loading.

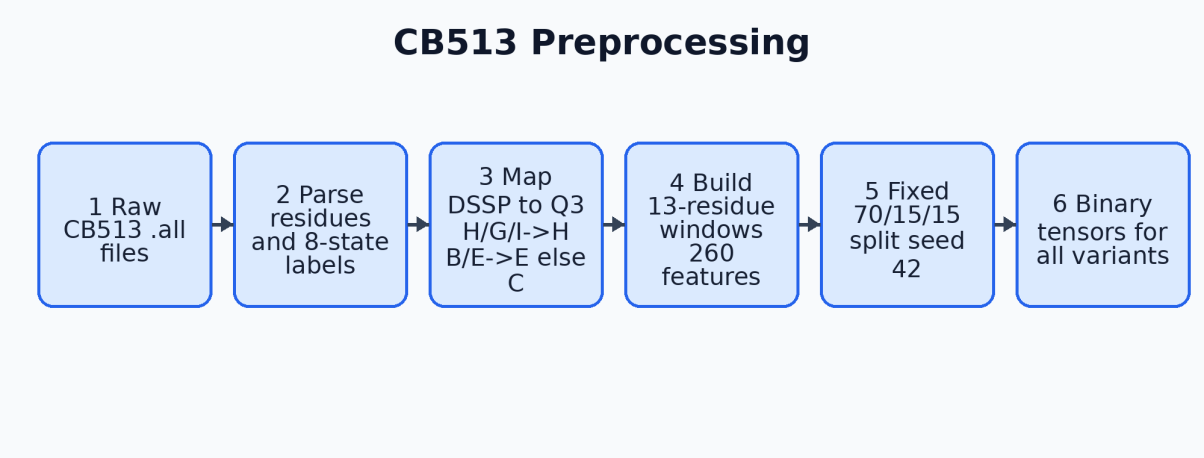

Figure 1: Preprocessing workflow. Raw CB513 files are parsed, mapped from eight DSSP classes to three Q3 classes, converted to sliding-window one-hot features, split, and persisted as binary tensors for all training variants.

Final sample counts are summarised in Table 1. The class distribution is approximately 33% H / 22% E / 45% C across every split, consistent with published statistics for CB513.

Table 1: Dataset splits after preprocessing.

Split	Proteins	Residues (samples)	Feature dim	Classes
train	359	58 363	260	3
val	77	13 064	260	3
test	77	12 692	260	3

3 Serial Model and the Q_3 Metric

The model is a two-hidden-layer MLP:

$$\text{Input}(260) \to \text{Dense}(H_1) \to \text{ReLU} \to \text{Dense}(H_2) \to \text{ReLU} \to \text{Dense}(3) \to \text{Softmax}.$$

Loss is cross-entropy fused with softmax for a numerically stable backward pass:

$$\frac{\partial L}{\partial z_3} = p - \mathbf{1}_{y_{\text{true}}},$$

where $p = \text{softmax}(z_3)$ and $\mathbf{1}_y$ is the one-hot encoding of the true label. Optimisation uses mini-batch SGD with an optional step-decay learning rate. The reference configuration used for every comparison run is

$$H_1 = 256,\ H_2 = 128,\ \text{epochs} = 80,\ \text{batch} = 64,\ \text{lr} = 0.01,\ \text{seed} = 42.$$

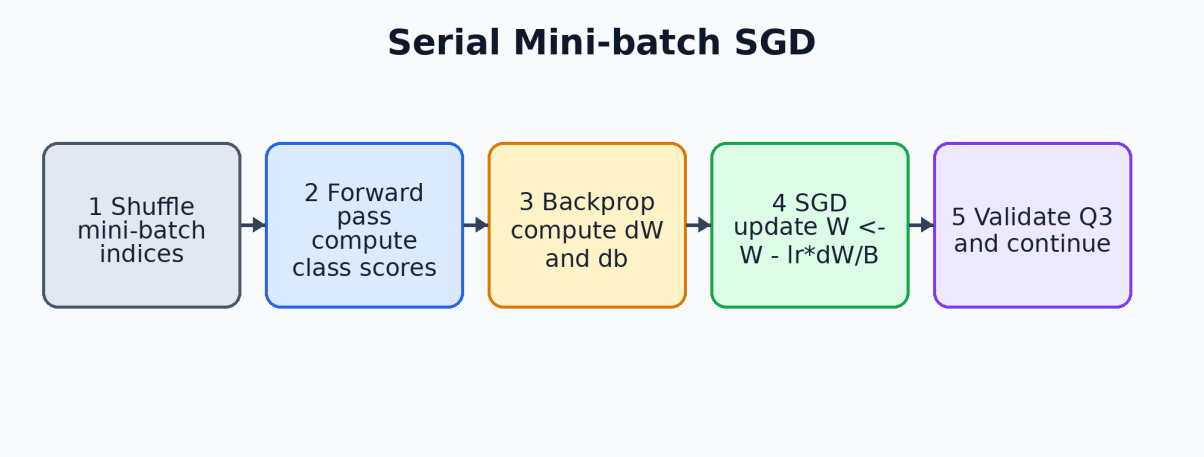

Figure 2: Serial training-loop workflow. Each mini-batch performs forward propagation, back-propagation, one SGD update, validation logging, and final test evaluation.

The Q_3 metric, following Eq. (19) of Zhong et al., is

$$Q_3 = \frac{N_H^{\text{correct}} + N_E^{\text{correct}} + N_C^{\text{correct}}}{N_{\text{total}}} \times 100\,\%. \tag{1}$$

This project is a three-class classification problem, so Q3 accuracy is the appropriate serial-vs-parallel accuracy metric rather than a regression metric such as RMSE.

On the reference 80-epoch run (single CPU worker, GCC with `-O3 -march=native`) the serial baseline achieves

$$\boxed{\text{test } Q_3 = 59.28\,\%} \qquad \text{total training time} = 236.8\,\text{s}.$$

All parallel variants are validated against this reference.

4 Parallel Designs

Every variant trains the same MLP and uses the same mini-batch SGD semantics. The variants differ only in how the mini-batch is partitioned among workers and how their per-worker gradients are combined before the SGD update. A single rule holds across all designs:

> *Model weights are **read-only** during the parallel region. Each worker accumulates gradients into a **private** buffer; a reduction sums these; a single SGD update is then applied. This removes all locking on the weight matrices.*

4.1 OpenMP

The OpenMP variant uses shared-memory data parallelism. The model weights are shared but read-only while a mini-batch is split across threads. Each thread computes forward and backward passes for its slice, writes private gradients, and then the master thread reduces those gradients and applies one SGD weight update.

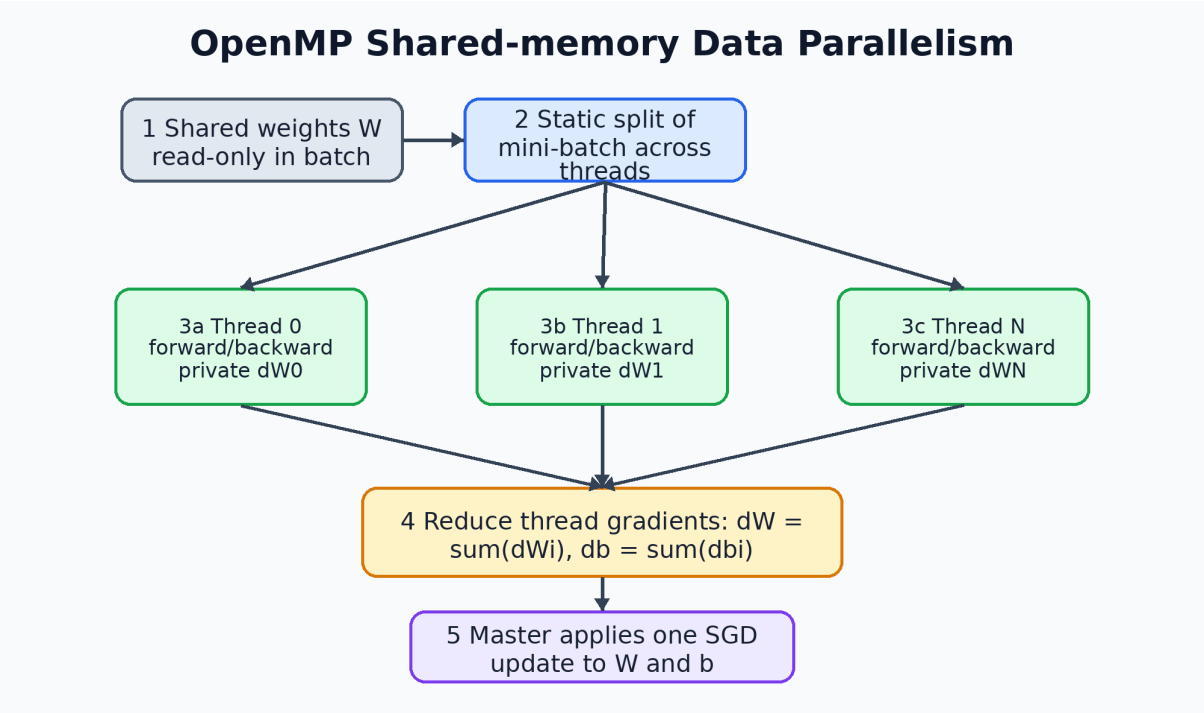

Figure 3: OpenMP shared-memory workflow. Model weights are shared and read-only during the parallel region; each thread writes only to its own private gradient buffer before the master performs one reduction and SGD update.

Runtime placement was pinned with `OMP_PROC_BIND=true OMP_PLACES=cores` for every timing run.

4.2 POSIX Threads

The POSIX Threads variant implements the same shared-memory idea manually with a persistent master/worker pool. The master publishes a mini-batch, workers process their assigned slices into private gradient buffers, a barrier waits for all workers, and the master reduces the gradients before the single SGD update.

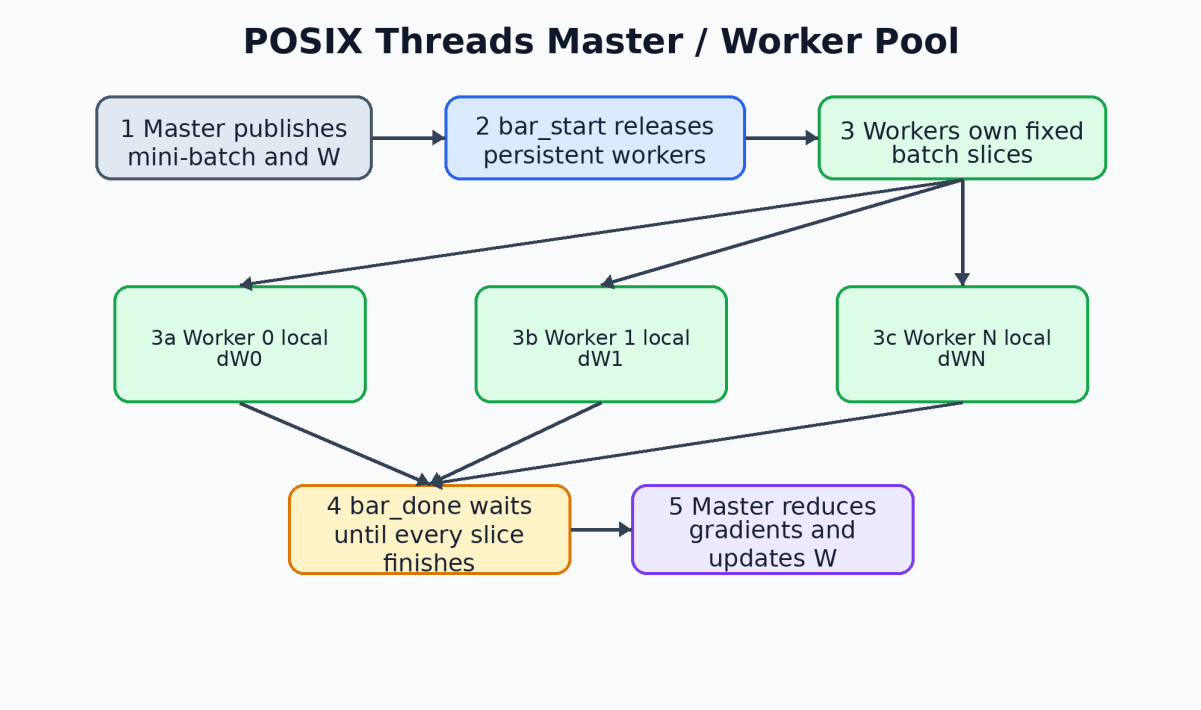

Figure 4: Pthreads master/worker workflow. Workers live for the entire run; two barriers gate each mini-batch, and the master performs the gradient reduction and SGD update after all workers finish their slices.

Shutdown is clean: the master sets an `exit_flag`, trips `bar_start` one last time, and each worker returns and is `pthread_join`-ed.

4.3 MPI (Distributed Memory)

The MPI variant runs the same algorithm across ranks with separate address spaces. It can run across physical machines, but the final measurements in this report use local multi-rank execution on the HPC host. All ranks load the same data split and initialise the same model state, so they agree on the mini-batch order.

Per mini-batch, rank r out of R processes a contiguous slice $[r \cdot \text{chunk}, (r{+}1) \cdot \text{chunk})$ of the batch and accumulates into a *rank-local* gradient. The six gradient arrays are backed by a **single contiguous flat buffer** containing the weight and bias gradients. One packed `MPI_Allreduce` therefore handles the whole gradient state in a single collective; six separate collective calls would pay latency six times. After the Allreduce every rank applies the same SGD update, keeping the models bit-identically synchronised. Validation, test inference, and JSON logging happen only on rank 0.

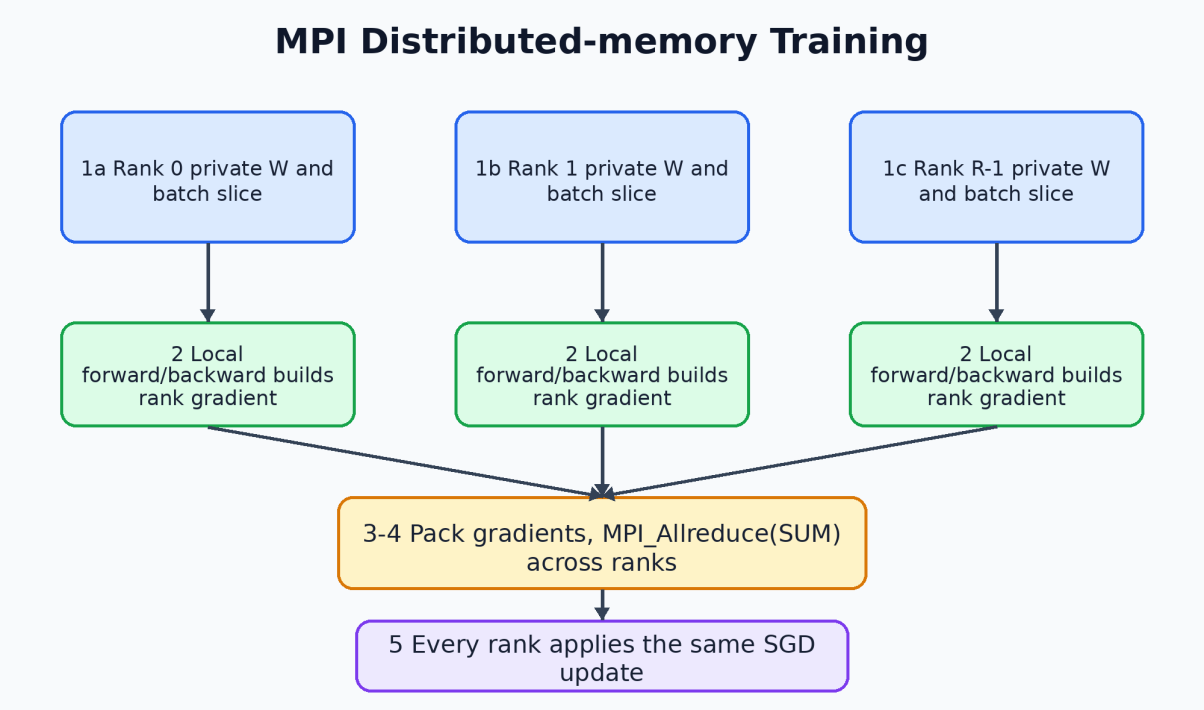

Figure 5: MPI distributed-memory workflow. Each rank owns a private model copy and local gradient; one packed `MPI_Allreduce(SUM)` synchronises the full gradient before every rank applies the same SGD update.

Timing uses `MPI_Barrier` around the epoch timer and an `MPI_Reduce(MAX)` on per-rank epoch times so that the reported number reflects the slowest rank.

4.4 Hybrid MPI + OpenMP

The hybrid variant combines both layers of parallelism:

- **Outer (MPI)** — identical to Section 4.3: ranks split the mini-batch across machines and combine their partial gradients with `MPI_Allreduce`.
- **Inner (OpenMP)** — inside each rank, the rank's slice is further split across OpenMP threads using the same per-thread `MLPGrad` pattern as Section 4.1. The intra-rank reduction is performed before the cross-rank Allreduce.

We initialise MPI with `MPI_Init_thread(MPI_THREAD_FUNNELED)` — only the main thread makes MPI calls, and indeed all our MPI collectives are *outside* OpenMP regions. The oversubscription rule is ranks $\times$ threads $\leq$ cores per machine. We pin OpenMP threads to CPU cores during timing runs.

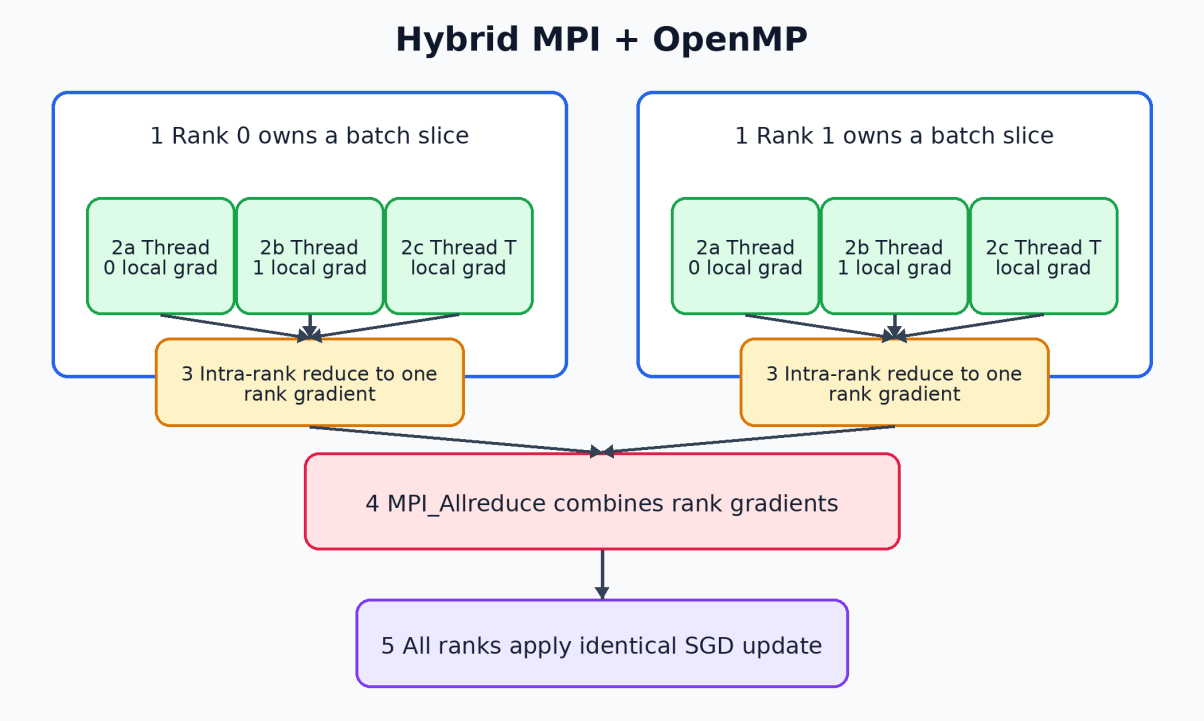

Figure 6: Hybrid MPI+OpenMP workflow. MPI performs the outer distributed-memory decomposition across ranks, OpenMP performs the inner shared-memory decomposition within each rank, and only the rank-local reduced gradients enter the MPI `Allreduce`.

4.5 CUDA

The CUDA variant keeps the training tensors and weights on the GPU. The design decisions are:

- **Data resident on device.** Full train/val/test tensors and the shuffled index list are uploaded once; per-batch cost is only a small `cudaMemcpy` of the batch-index slice.
- **Column-major throughout** — cuBLAS native; a single gather kernel builds a column-major batch matrix from the row-major device copy using the index slice, avoiding a global transpose.
- **cuBLAS `sgemm` for every GEMM** — forward $W \cdot X$, gradient $\mathrm{d}W = \mathrm{d}Z \cdot A^\top$, and back-propagation $\mathrm{d}A = W^\top \mathrm{d}Z$.
- **Fused gradient + SGD step.** Instead of materialising explicit gradient buffers on the device, we call the gradient-producing `sgemm` with $\alpha = -\mathrm{lr}/B$ and $\beta = 1$, which lets the GEMM write directly into the weight matrix. Bias updates use `sgemv` against a vector of ones with the same scaling.
- **Custom kernels** handle the three element-wise ops: `bias_relu_kernel` (bias-add followed by ReLU), `relu_mask_kernel` (ReLU derivative on the back-propagated gradient), and a fused `softmax_xent_grad_kernel` which, per batch column, applies b_3, computes numerically stable softmax, accumulates the cross-entropy loss via `atomicAdd`, and converts Z_3 in place to $\mathrm{d}Z_3 = p - \mathbf{1}_y$.
- **Validation / test Q3** copies the trained weights back to the host only when CPU-side evaluation is needed.

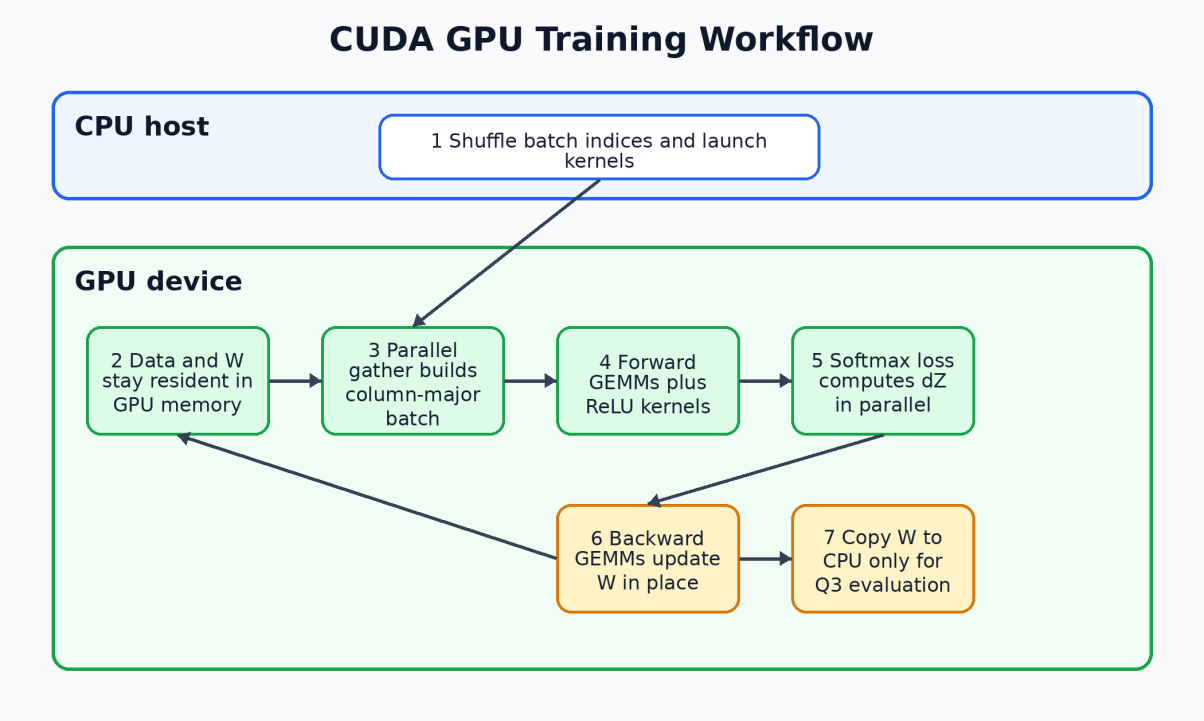

Figure 7: CUDA GPU workflow. The host shuffles indices and launches kernels, while the training data, model tensors, cuBLAS operations, custom kernels, and in-place SGD updates remain on the GPU.

5 Experimental Setup

Execution host. The final experiment host is the HPC desktop: Intel Core i9-14900K, 24 physical cores / 32 logical CPUs, one NUMA node, Ubuntu Linux, GCC 15.2, and `-O3 -march=native`. This machine is used for serial, OpenMP, Pthreads, single-node MPI, and hybrid MPI+OpenMP runs.

MPI mode. MPI is executed locally with multiple ranks on the same HPC host. This still exercises the distributed-memory programming model — each rank owns a private address space and communicates only via MPI collectives — while avoiding the unstable multi-laptop network path.

GPU host. CUDA is run on the same HPC machine when launched from a shell that can access `/dev/nvidia*`. The detected GPU is an NVIDIA GeForce RTX 5090 with 32 GB VRAM, driver 595.71.05, and CUDA runtime support reported by `nvidia-smi`. The installed compiler is `nvcc` 12.4 and cuBLAS is linked by the CUDA build.

Fairness controls.

- Same `-seed 42` for every run ⇒ identical initial weights and per-epoch shuffle order.
- Same hyperparameters ($H_1 = 256$, $H_2 = 128$, lr $= 0.01$, batch $= 64$).
- Same 70/15/15 split.
- `OMP_PROC_BIND=true OMP_PLACES=cores` for every shared-memory run.
- Timing excludes data loading and final evaluation; it covers only the epoch loop.

Measurement protocol.

- **Accuracy table** — full 80-epoch run per variant at the reference configuration. CPU variants must stay within ±0.5 percentage points of serial test Q_3; CUDA is accepted within ±1.0 point because GPU reduction order differs.
- **Timing sweep** — 20-epoch run per configuration, repeated three times. The reported timing value is the mean of the per-epoch `epoch_time_s` values in the JSON log, with standard deviation across repeated runs.

6 Results

6.1 Accuracy parity with the serial baseline

Table 2 summarises the test Q_3 reached by every variant at the reference configuration. Parallelism is expected to improve runtime, not accuracy: OpenMP, Pthreads, MPI, and Hybrid execute the same SGD update after summing private gradients, so their Q3 should match the serial trajectory within CPU floating-point tolerance. CUDA can drift by up to ±1% because cuBLAS fuses and reorders float reductions, which does not change the underlying algorithm but can change the per-weight rounding trajectory.

Table 2: Final test Q_3 by variant at the reference configuration.

Variant	Configuration	Test Q_3	Δ vs serial	Total time
serial	1 thread	**59.28 %**	— (reference)	236.8 s
openmp	16 threads	59.42 %	+0.14 pp	95.0 s
pthreads	16 threads	59.23 %	-0.06 pp	87.2 s
mpi	4 ranks	59.23 %	-0.06 pp	85.8 s
hybrid	4 ranks × 4 threads	59.30 %	+0.02 pp	78.0 s
cuda	batch 64, RTX 5090	59.37 %	+0.09 pp	6.3 s

Figure 8 visualises the same final test accuracy comparison. The dashed line marks the serial baseline; every parallel implementation remains within the accepted tolerance, so the runtime gains in the following sections are not obtained by changing the classification outcome.

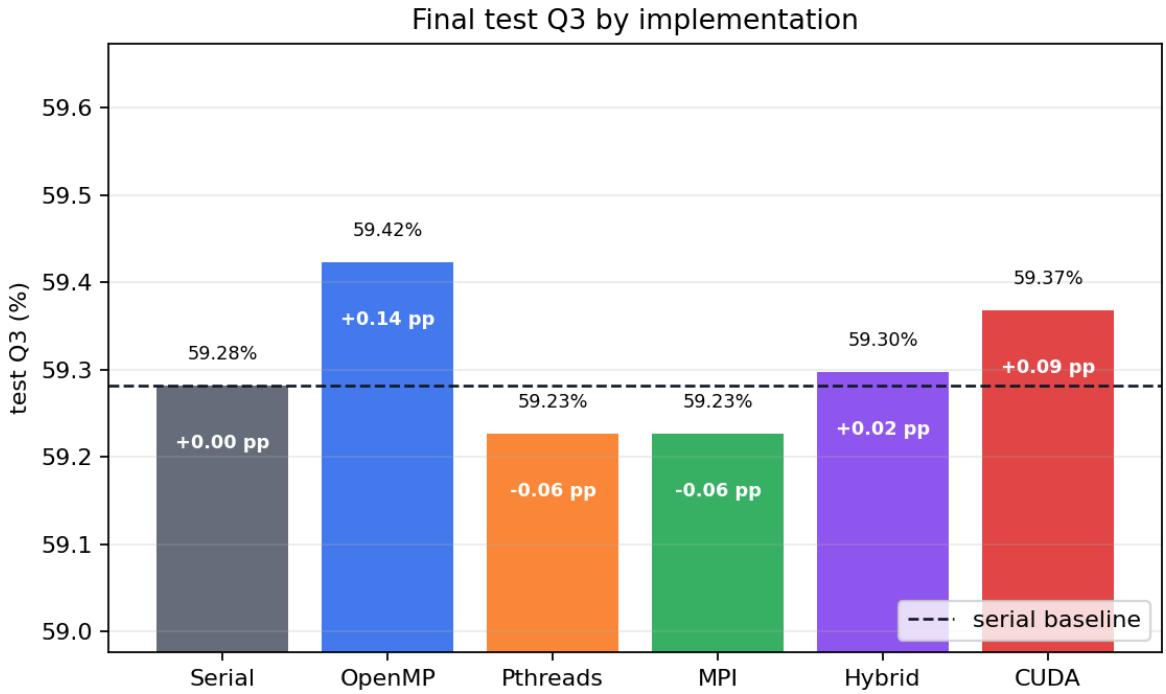

Figure 8: Final test Q_3 by implementation. Bar labels show each variant's absolute Q3 and the percentage-point difference from the serial baseline.

Figure 9 compares the 80-epoch training trajectories. The validation-Q3 and training-loss curves overlap closely, which confirms that the CPU and GPU parallelisations preserve the serial mini-batch SGD learning path up to normal floating-point ordering effects.

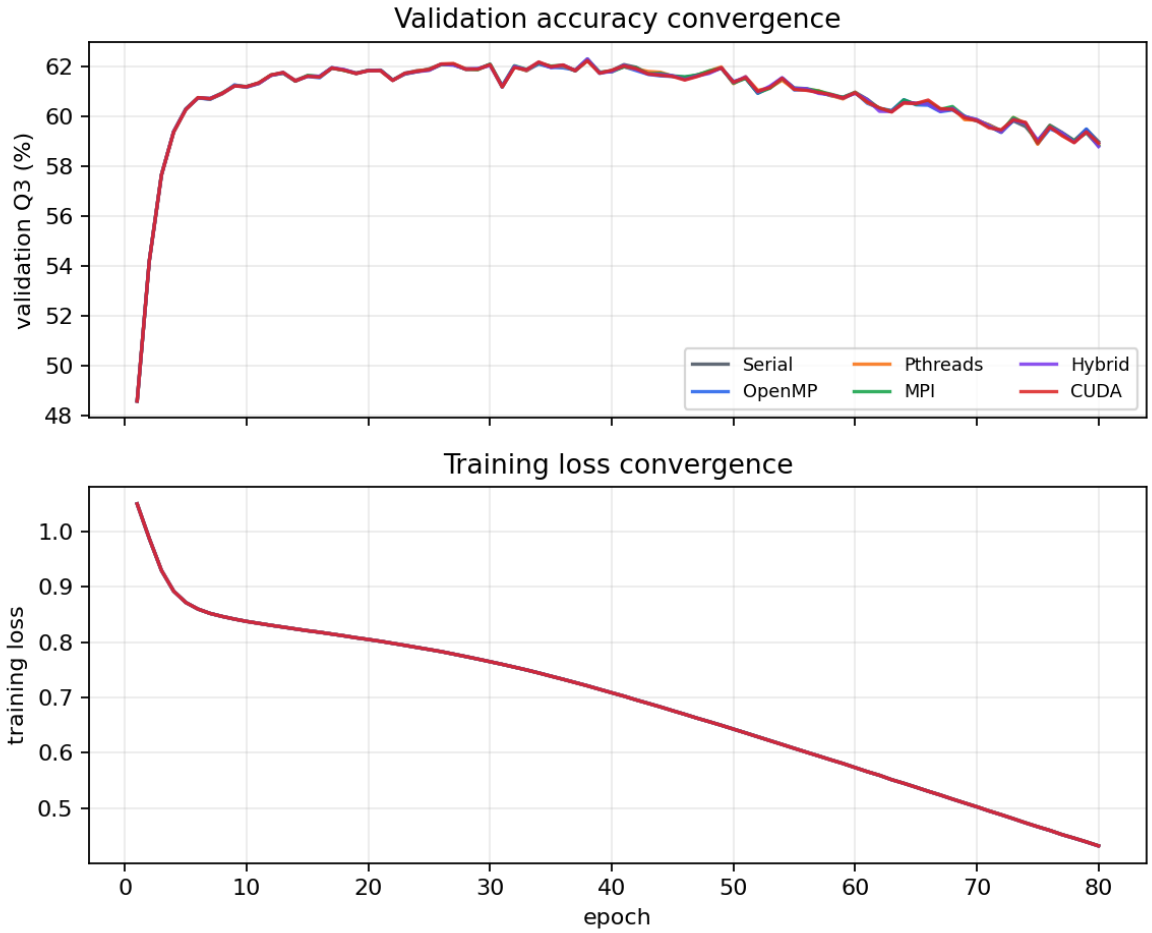

Figure 9: Training convergence for the six reference implementations. The upper panel shows validation Q_3 over epochs; the lower panel shows training loss over epochs.

6.2 Shared-memory scaling: OpenMP vs Pthreads

Figure 10 reports the per-epoch wall-clock time. Figure 11 reports the resulting speedup relative to the single-threaded run of the same variant.

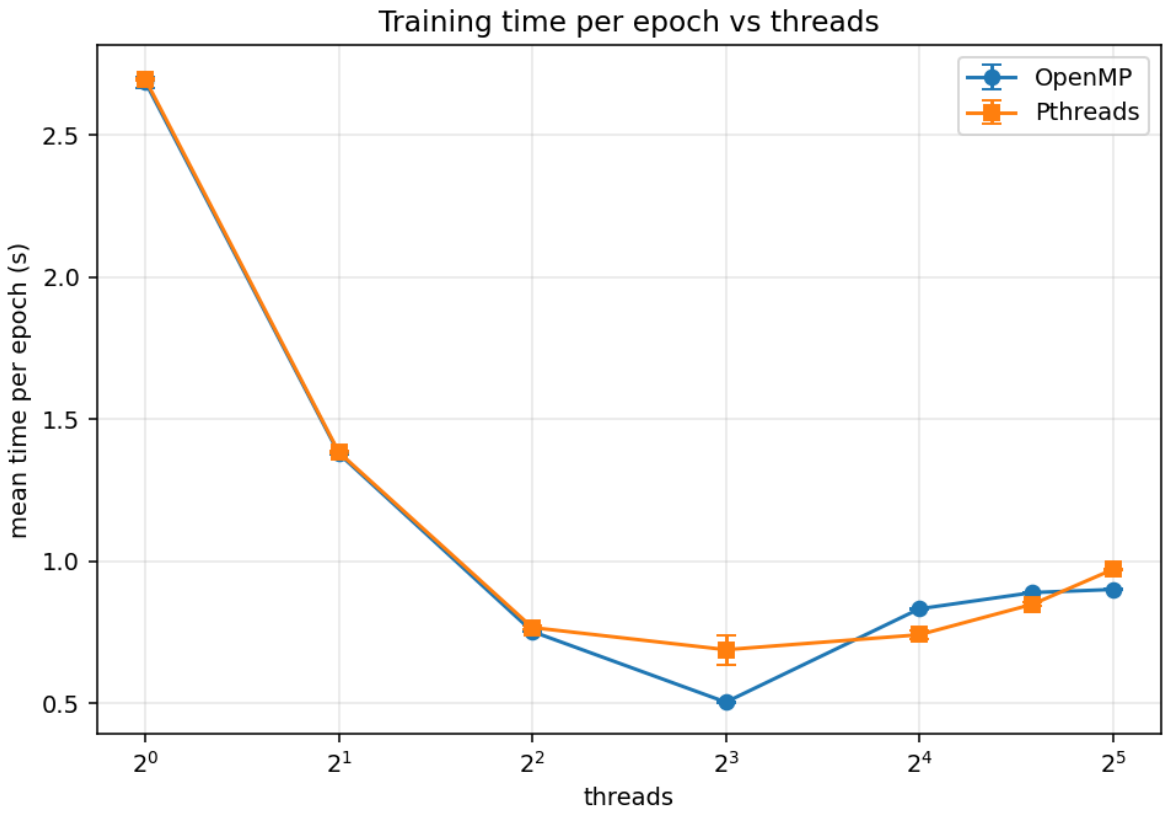

Figure 10: Shared-memory training time per epoch. Both variants use the same mini-batch data-parallel reduction scheme.

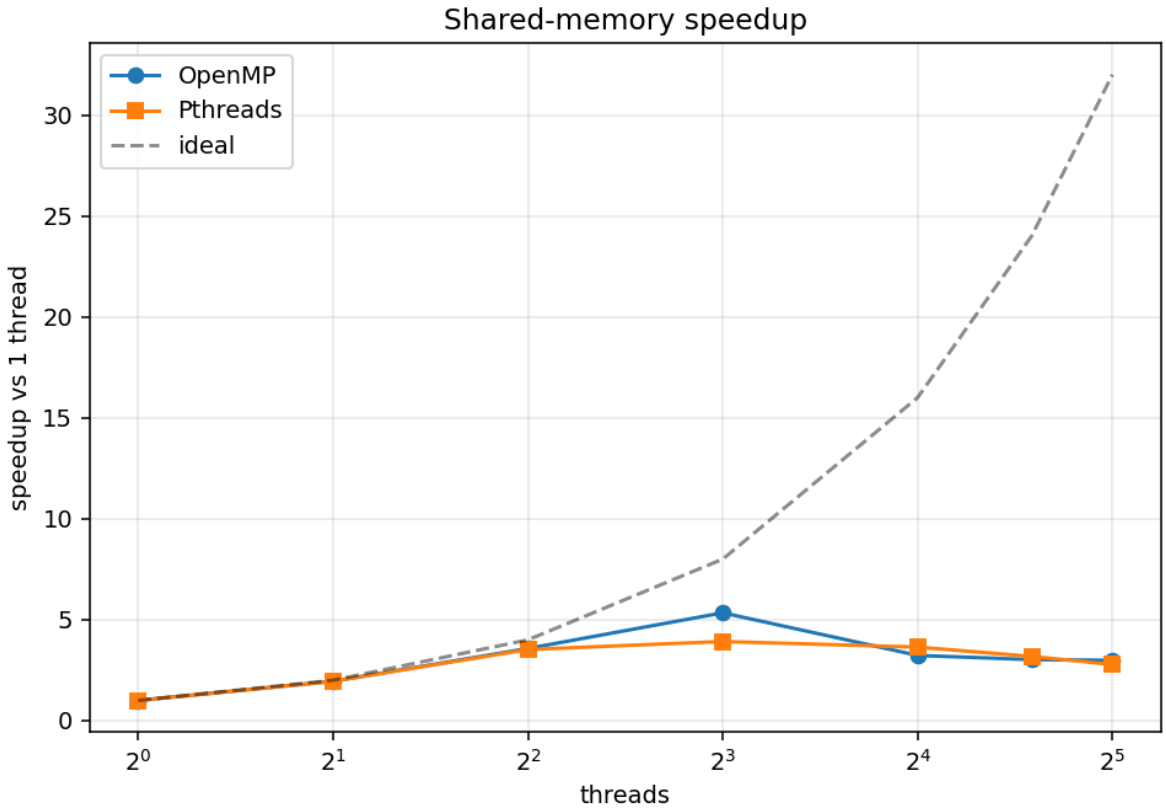

Figure 11: Shared-memory speedup relative to each variant's one-thread run.

The 20-epoch timing sweep gives:

- **OpenMP** improves from 2.687 s/epoch at one thread to a best 0.503 s/epoch at eight threads, a 5.34× speedup. Beyond eight threads, runtime rises to 0.832 s/epoch at 16 threads and 0.900 s/epoch at 32 threads.
- **Pthreads** improves from 2.696 s/epoch at one thread to a best 0.688 s/epoch at eight threads, a 3.92× speedup. The persistent worker-pool design avoids thread creation overhead, but the two-barrier dispatch still costs more than OpenMP's runtime team here.
- **Diminishing returns after eight workers** are consistent with the small MLP and per-batch gradient reduction cost: once each worker has too little sample work, synchronisation and cache effects dominate the useful forward/backward computation.

6.3 Distributed-memory scaling: MPI

Figure 12 shows the per-epoch time for local MPI rank counts on the HPC host. The accuracy at every rank count is identical (to within the float tolerance in Section 6.1) because the Allreduce(SUM) reproduces the single-rank gradient exactly.

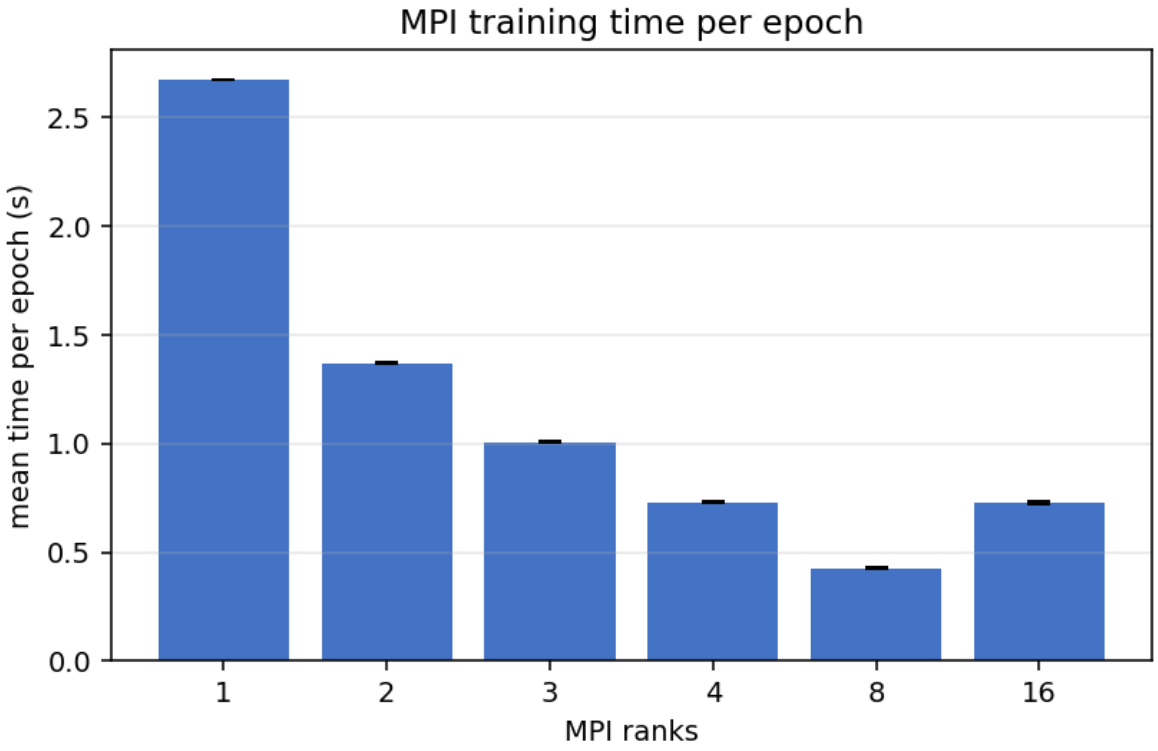

Figure 12: MPI training time per epoch on the HPC host, using separate MPI address spaces and `Allreduce` communication.

Per-rank compute drops approximately $1/R$, but the collective synchronisation cost of the per-batch `Allreduce` on the packed gradient buffer (~ 250 kfloats) is fixed. On this host, MPI improves from 2.677 s/epoch at one rank to a best 0.427 s/epoch at eight ranks (6.26×), then worsens to 0.729 s/epoch at 16 ranks. This identifies eight local ranks as the useful MPI point for this model and batch size.

6.4 Hybrid MPI + OpenMP

Figure 13 reports the hybrid configuration sweep on the HPC host. The tested layouts include $1{\times}1, 1{\times}8, 1{\times}16, 1{\times}24, 2{\times}4, 2{\times}8, 4{\times}4, 4{\times}8, 8{\times}2, 8{\times}4$, all constrained by $R \cdot T \leq 32$. The design motivation is that `MPI_Allreduce` is not used inside a node — that work is off-loaded to OpenMP's thread-local reduction — so only inter-node gradient exchange pays the network cost.

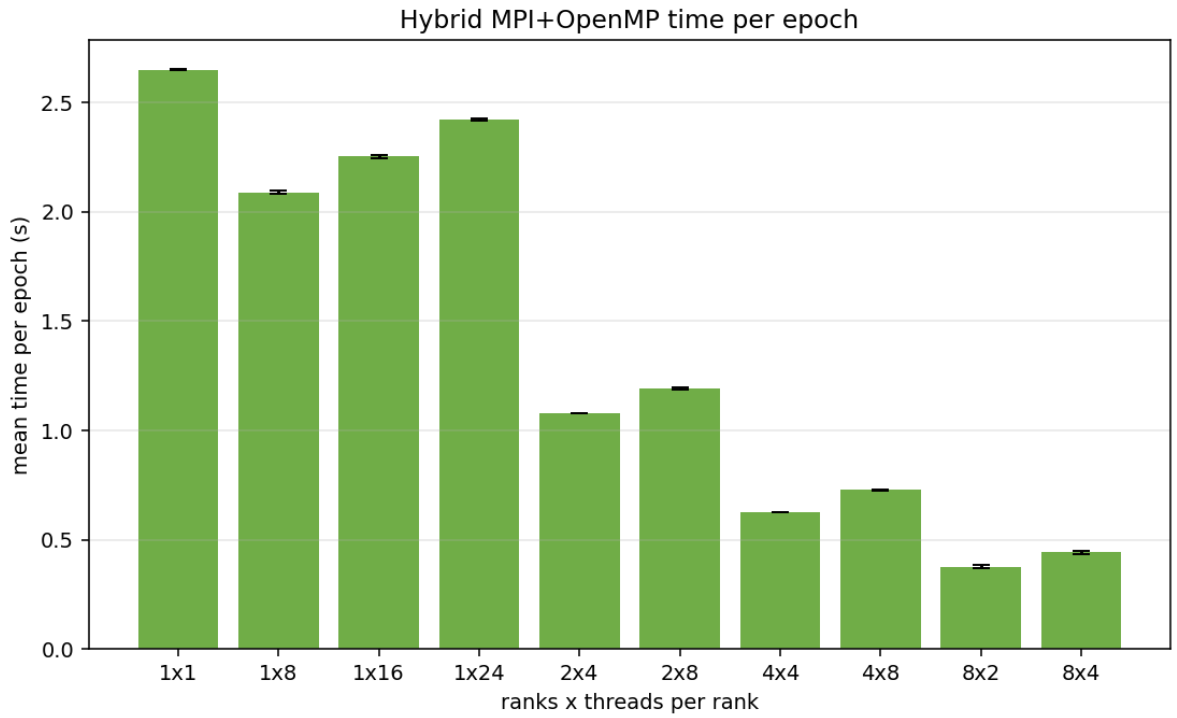

Figure 13: Hybrid per-epoch time over the local (R, T) matrix.

The fastest hybrid timing point is 8×2 (eight MPI ranks, two OpenMP threads per rank), at 0.378 s/epoch. This is faster than pure MPI at eight ranks (0.427 s/epoch) and much faster than pure OpenMP at eight threads (0.503 s/epoch), showing that the nested decomposition reduces per-rank sample work without pushing the OpenMP thread count into its slower high-thread regime. The 80-epoch accuracy run uses the more balanced 4×4 layout and still matches serial within 0.02 percentage points.

6.5 GPU: CUDA

Figure 14 plots per-epoch time against mini-batch size on the RTX 5090.

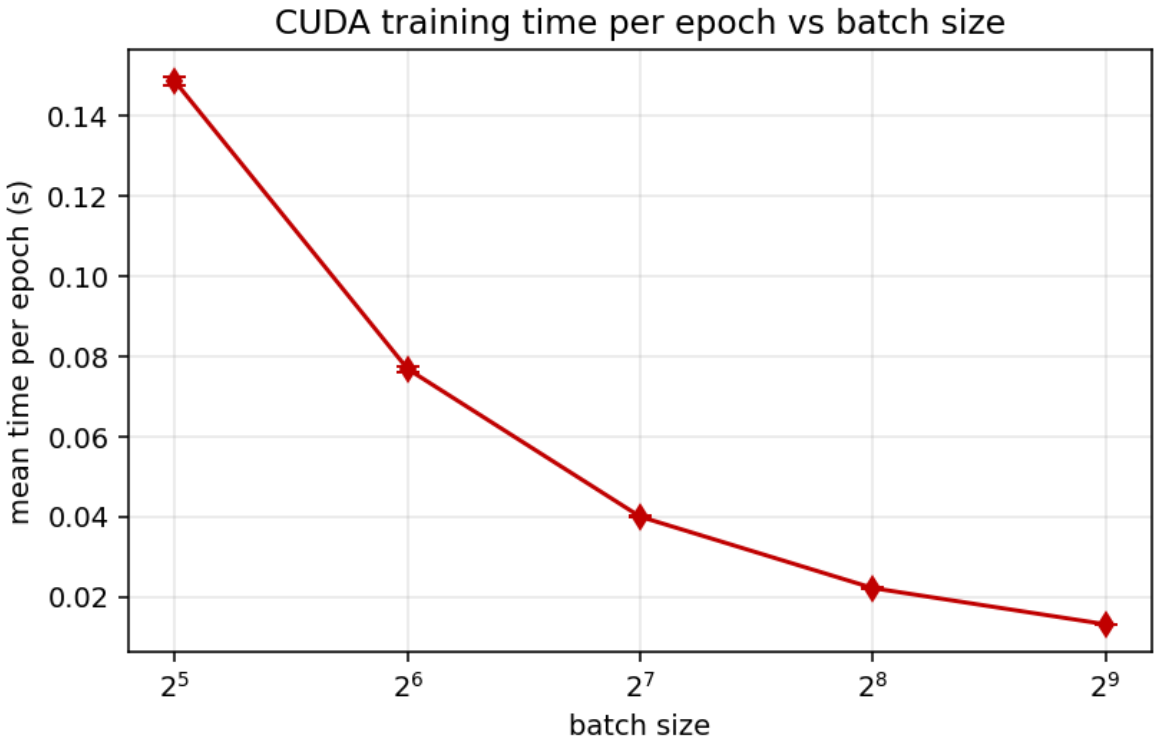

Figure 14: CUDA training time per epoch vs mini-batch size.

CUDA is the fastest implementation by a large margin. At the reference batch size 64 it runs at 0.0769 s/epoch and completes the 80-epoch accuracy run in 6.3 s, about 37.8× faster than the serial 80-epoch baseline. Larger batches reduce per-epoch time further (0.0401 s at batch 128, 0.0223 s at 256, and 0.0132 s at 512), but they also change the optimisation path and reduce the 20-epoch test Q3 from 62.28 % at batch 64 to 56.98 % at batch 512. For the accuracy comparison, batch 64 is therefore the fair CUDA setting.

7 Discussion

7.1 Where scaling works well

Mini-batch-level data parallelism is the natural primitive. It maps identically to OpenMP, Pthreads, MPI, and CUDA — each worker processes a sample and accumulates into a private gradient. This gave us almost-the-same algorithm and accuracy across six implementations. Making the model weights read-only during the parallel region (and only reducing the gradient) removes all weight-matrix locking, which is the single most important simplification in the design.

7.2 Where scaling plateaus

- **Small model, small per-sample work.** One forward/backward pass through the 260–256–128–3 MLP is cheap. As we add threads, the fixed reduction cost of about 250 kfloats per batch consumes a growing share.
- **Mixed CPU topology and SMT.** On the i9-14900K, scaling changes as runs move from performance cores to efficiency cores and then to SMT siblings. SMT threads share execution units and caches with their physical twin, so the final scaling step is naturally sub-linear.

- **MPI `Allreduce` overhead.** With $\sim 250\,\text{kfloat} \times 4\,\text{B} = 1\,\text{MB}$ per Allreduce, and several hundred batches per epoch, collective overhead is visible even on a single host. On multiple physical machines this same cost would also include network latency. We mitigated it by packing all six gradient arrays into one flat buffer (one Allreduce per batch instead of six). Further reduction would require gradient accumulation across batches (changing the optimisation semantics) or overlap of Allreduce with the next batch's forward pass.

7.3 Why OpenMP and Pthreads can differ

Both implementations use the *same* data-parallel reduction scheme, so at steady state they converge to near-identical throughput. Any differences come from **runtime overhead**:

- OpenMP keeps a thread team alive across parallel regions, so recurring parallel work reuses the same workers.
- Our Pthreads implementation is *also* a persistent pool with barrier-based dispatch — that was a deliberate design choice to avoid per-batch fork/join overhead. A naive thread-create per batch is orders of magnitude slower.
- At very low thread counts (1–2) the OpenMP runtime's first-touch memory placement tends to be slightly better, giving OpenMP a small edge. At higher thread counts the gap closes and either variant's overhead is dwarfed by the reduction cost.

7.4 Practical limits from our hardware

- The MPI results are single-node multi-rank results. They validate the distributed-memory code path, but do not include real network latency between physical machines.
- The CPU has mixed performance and efficiency cores. Scaling curves should therefore be interpreted by core class: moving from P-cores to E-cores and then to SMT threads naturally changes the slope.
- CUDA must be launched from a shell that can see `/dev/nvidia*`. Sandboxed shells may fail even when the host terminal's `nvidia-smi` works.

References

[1] W. Zhong, G. Altun, X. Tian, R. Harrison, P.C. Tai, Y. Pan. *Parallel protein secondary structure prediction schemes using Pthread and OpenMP over hyper-threading technology.* J. Supercomputing, 41(1):1–16, 2007.

[2] J.A. Cuff, G.J. Barton. *Evaluation and improvement of multiple sequence methods for protein secondary structure prediction.* Proteins, 34(4):508–519, 1999.

[3] B. Rost, C. Sander. *Improved prediction of protein secondary structure by use of sequence profiles and neural networks.* PNAS, 90:7558–7562, 1993.

[4] Open MPI Project, version 5.x user manual. `https://www.open-mpi.org/doc/v4.1/`

[5] NVIDIA cuBLAS Library documentation, CUDA Toolkit 12.x. `https://docs.nvidia.com/cuda/cublas/`